



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MACHINE LEARNING – ITA0603

COURSE ASSIGNMENT REPORT

**Development and Empirical Evaluation of High-Scale Predictive Maintenance and Asset
Optimization Infrastructure using Hybrid Machine Learning Paradigms**

Submitted by

NIVETHA M

Register No.: 192421307

Academic Year: 2026

ABSTRACT

This report presents an integrated machine learning framework for high-scale predictive maintenance and asset optimization in sustainable industrial infrastructure. The framework progresses from symbolic concept learning to connectionist, probabilistic, instance-based, and evolutionary learning paradigms. Part A investigates Candidate Elimination and ID3 decision-tree learning, including version-space behavior under sensor noise and asymptotic complexity. Part B develops a Perceptron, Multilayer Perceptron (MLP) with backpropagation, Expectation-Maximization (EM), and a Minimum Description Length (MDL) comparison with a Bayesian model. Part C combines Locally Weighted Regression (LWR), Naïve Bayes, and a Genetic Algorithm (GA) into a hybrid pipeline. Synthetic telemetry data are generated and benchmarked at 10^3 , 10^4 , 10^5 , and 10^6 records. The resulting architecture demonstrates how model selection, local smoothing, probabilistic classification, and evolutionary hyperparameter optimization can support scalable and resource-aware infrastructure analytics.

1. INTRODUCTION

The assignment addresses the need for robust machine learning in infrastructure systems that produce large volumes of spatial, environmental, and operational telemetry. Such systems may include sustainable industrial facilities, smart grids, or communication hubs. The source assignment specifies a scale from 10^3 to 10^6 records and requires the entire solution to be implemented and evaluated in a common computational environment.

The solution is organized into three parts. Part A focuses on computational diagnostics and inductive bias. Part B evaluates connectionist and probabilistic architectures. Part C creates a hybrid framework that combines instance-based learning, probabilistic classification, and genetic optimization. This organization follows the assignment framework.

2. AIM

To design, implement, and empirically evaluate a scalable hybrid machine learning framework for predictive maintenance and anomaly detection in high-dimensional infrastructure telemetry, while analyzing computational complexity, model behavior, and sustainable deployment.

3. OBJECTIVES

- Implement Candidate Elimination and analyze Version Space boundaries under sensor noise.
- Implement ID3-style decision-tree learning using entropy and information gain.
- Compare restriction bias and search bias in concept learning.
- Analyze the theoretical time and space complexity of the learning methods.
- Implement a Perceptron and an MLP trained using backpropagation.
- Study error convergence and the possibility of local-minimum behavior during neural-network training.
- Implement EM for estimating hidden parameters in a probabilistic model with incomplete observations.

- Apply the MDL principle to compare model fit and structural complexity.
- Implement LWR for local smoothing and Naïve Bayes for anomaly classification.
- Use a Genetic Algorithm to optimize the instance-based parameter k.
- Benchmark the scalable pipeline from 10^3 to 10^6 records.
- Explain the relationship between computational efficiency and SDG 9 Target 9.4.

4. PROBLEM STATEMENT

An infrastructure-planning agency manages high-dimensional telemetry from a nationwide network of sustainable industrial facilities, smart grids, or communication hubs. The data contain large collections of environmental and operational sensor records. The agency must minimize energy consumption, prevent structural redundancies, and optimize predictive maintenance under noisy and incomplete sensor observations. A multi-paradigm machine learning architecture is therefore required, moving from strict rule-based concept learning to neural, probabilistic, instance-based, and evolutionary methods.

5. SYSTEM ARCHITECTURE

The proposed system follows the pipeline below:

Raw Telemetry → Data Cleaning → Candidate/Concept Analysis → LWR Local Smoothing → Naïve Bayes Anomaly Classification → GA Hyperparameter Optimization → Performance Benchmarking → Sustainable Deployment Decision

In parallel, the Perceptron/MLP branch models structural stress, while the EM branch estimates hidden probabilistic parameters from incomplete observations. MDL is used as a model-selection criterion.

PART A – COMPUTATIONAL DIAGNOSTICS & INDUCTIVE BIAS ANALYSIS

6. Candidate Elimination and Version Space

Candidate Elimination maintains two boundaries of the Version Space: the specific boundary S and the general boundary G. S contains maximally specific hypotheses consistent with the training examples, whereas G contains maximally general consistent hypotheses. A positive example requires hypotheses to be generalized, when necessary, while a negative example requires overly general hypotheses to be specialized.

For a hypothesis h and example x, the coverage condition is:

$h \text{ covers } x \Leftrightarrow \forall i, h_i = ? \text{ or } h_i = x_i$

The Version Space is the set of all hypotheses that are consistent with every observed training example:

$VS(H,D) = \{ h \in H \mid \forall (x,y) \in D, h(x) = y \}$

7. Version Space Collapse Under Noise

Under the standard consistent concept-learning assumption, Candidate Elimination requires at least one hypothesis that agrees with every labeled example. If contradictory observations are introduced, the Version Space may become empty. For example, suppose two identical feature vectors x receive opposite labels:

$$D = \{(x, +), (x, -)\}$$

Any deterministic hypothesis must assign one label to x . Therefore no hypothesis can satisfy both constraints simultaneously and:

$$VS(H,D) = \emptyset$$

At scale, random sensor faults can create many such inconsistencies. Thus the S and G boundaries may eventually become incompatible, producing a null Version Space. This is a fundamental limitation of strict consistency-based concept learning in noisy environments.

8. Candidate Elimination Algorithm

1. Initialize S to the most specific hypothesis and G to the most general hypothesis.
2. Read each training example sequentially.
3. For a positive example, remove hypotheses from G that do not cover the example and minimally generalize S .
4. For a negative example, remove hypotheses from S that cover the example and minimally specialize G .
5. Remove hypotheses that are inconsistent with the boundaries.
6. Continue until all examples are processed.
7. The remaining S and G boundaries define the Version Space.

9. ID3 Decision Tree Learning

ID3 selects attributes using information gain. Entropy measures the uncertainty of the target variable. For a dataset D with class probabilities p_i :

$$H(D) = - \sum_i p_i \log_2(p_i)$$

For attribute A with values v , the expected conditional entropy is:

$$H(D|A) = \sum_v |D_v|/|D| \times H(D_v)$$

Information Gain is:

$$IG(D,A) = H(D) - H(D|A)$$

The attribute with the highest information gain is selected as the decision node. Unlike Candidate Elimination, ID3 uses a heuristic search preference over the available hypothesis representation rather than enforcing a strict consistency boundary.

10. Inductive Bias Comparison

Method	Main Bias	Behavior	Noise Sensitivity
Candidate Elimination	Restriction / representation bias	Maintains all consistent hypotheses	High under contradictory labels
ID3	Search / preference bias	Greedily selects high-information attributes	Moderate; can overfit without control

11. Complexity Analysis

Let n be the number of records, d the number of attributes, and $|H|$ the number of candidate hypotheses. A conceptual Candidate Elimination scan can be expressed as $O(n|H|)$ when hypothesis maintenance is considered explicitly. ID3 commonly requires approximately $O(nd \log n)$ for sorting-based continuous-feature implementations, while the simple information-gain scan is often described as $O(nd)$ per tree level.

Scale	Candidate Elimination	ID3	Naïve Bayes	LWR Query
10^3	$O(10^3 H)$	$O(10^3 d \log 10^3)$	$O(10^3 d)$	$O(10^3)$
10^4	$O(10^4 H)$	$O(10^4 d \log 10^4)$	$O(10^4 d)$	$O(10^4)$
10^5	$O(10^5 H)$	$O(10^5 d \log 10^5)$	$O(10^5 d)$	$O(10^5)$
10^6	$O(10^6 H)$	$O(10^6 d \log 10^6)$	$O(10^6 d)$	$O(10^6)$

The lower bound for a full data scan is $\Omega(n)$, because every record must be accessed at least once in a standard batch-learning setting. Hence many scalable preprocessing and classification operations are $\Theta(n)$ for fixed dimensionality d . The main scalability concern for Candidate Elimination is the growth of the effective hypothesis space.

PART B – CONNECTIONIST VS PROBABILISTIC ARCHITECTURE EVALUATION

12. Perceptron

The Perceptron is a single-layer linear classifier. For input vector x , weight vector w , and bias b , the activation is:

$$\hat{y} = \text{step}(w^T x + b)$$

For a misclassified training sample with target $y \in \{-1, +1\}$, the update is:

$$w \leftarrow w + \eta y x$$

$$b \leftarrow b + \eta y$$

The Perceptron works well when the classes are approximately linearly separable. Its limitation is that it cannot represent arbitrary nonlinear decision boundaries.

13. Multilayer Perceptron and Backpropagation

The MLP used in the implementation contains an input layer, hidden layers, and an output layer. The example network uses a 5–16–8–1 structure for stress prediction. The hidden layers use a nonlinear activation and the parameters are optimized by gradient descent through backpropagation.

For a neuron:

$$z = \sum_i w_i x_i + b$$

$$a = f(z)$$

For mean squared error with prediction $\hat{y}$ and target y :

$$L = (1/N) \sum (y - \hat{y})^2$$

Backpropagation applies the chain rule to calculate gradients for every parameter, followed by:

$$w \leftarrow w - \eta \partial L / \partial w$$

The loss-convergence curve is plotted by the program. Non-convex neural-network optimization can contain saddle points and local minima; initialization, learning rate, batch strategy, and network architecture influence convergence.

14. Expectation-Maximization (EM)

EM is an iterative procedure for estimating parameters when latent variables or incomplete observations are present. The implementation demonstrates a two-component Gaussian mixture.

For parameter θ and latent variable Z , the E-step computes:

$$\gamma_{ik} = P(Z_i=k \mid x_i, \theta)$$

The M-step updates the parameters using the responsibilities γ_{ik} . For Gaussian component k :

$$N_k = \sum_i \gamma_{ik}$$

$$\mu_k = (1/N_k) \sum_i \gamma_{ik} x_i$$

$$\sigma_k^2 = (1/N_k) \sum_i \gamma_{ik} (x_i - \mu_k)^2$$

$$\pi_k = N_k/N$$

The log-likelihood should generally improve or remain stable over EM iterations until convergence, subject to numerical implementation and initialization.

15. Minimum Description Length (MDL)

The assignment specifies the MDL cost as:

$$\text{MDL Cost} = -\log_2 P(D|h) + (|h|/2) \log_2 |D|$$

The first term rewards a model that explains the data well, while the second term penalizes model complexity. Therefore a model with slightly better training fit may still lose under MDL if it contains substantially more parameters.

For the implemented 5–16–8–1 MLP, the parameter count is:

$$5 \times 16 + 16 + 16 \times 8 + 8 + 8 \times 1 + 1 = 241 \text{ parameters}$$

The BBN comparison uses a compact parameter-count approximation in the implementation. The program reports both MDL values so that the model with the lower cost can be selected.

16. Part B Algorithms

8. Standardize the telemetry features.
9. Train the Perceptron and record classification errors per epoch.
10. Train the MLP using stochastic gradient descent and backpropagation.
11. Record the MLP loss curve and calculate test mean squared error.
12. Introduce missing observations for the EM demonstration.
13. Run E-step and M-step iterations until the parameter estimates stabilize.
14. Calculate the MDL cost for the MLP and the Bayesian model.
15. Select the model with the lower MDL cost as the more compact candidate.

PART C – HYBRID FRAMEWORK SYNTHESIS & GREEN INFRASTRUCTURE DEFENSE

17. Locally Weighted Regression

LWR gives greater importance to observations that are close to a query point. A Gaussian kernel is used:

$$w_i = \exp(-||x_i - x||^2 / (2\tau^2))$$

The locally weighted least-squares solution is:

$$\theta = (X^T W X)^{-1} X^T W y$$

The resulting local prediction smooths geographical or operational variations before anomaly classification.

18. Naïve Bayes Classifier

Naïve Bayes applies Bayes' theorem with a conditional independence assumption between features:

$$P(C|X) \propto P(C) \prod_i P(x_i|C)$$

For Gaussian features, the class-conditional density is estimated using the mean and variance of each feature for each class. The class with the highest posterior probability is selected.

19. Genetic Algorithm Optimization

The Genetic Algorithm searches for a useful value of k for instance-based routing. A population of candidate integer values is initialized. Fitness is measured from validation error. The best candidates are selected, combined through crossover, and occasionally mutated.

16. Initialize a population of candidate k values.
17. Evaluate each candidate using a local-neighbor classification fitness function.
18. Select the higher-fitness candidates.
19. Create new candidates through crossover.
20. Apply random mutation within the valid range.

21. Repeat for several generations.
22. Return the best-performing k.

20. Hybrid Pipeline

The final architecture integrates the methods rather than treating them as unrelated algorithms:

23. Collect raw telemetry and clean missing numerical values.
24. Use local regression to smooth local variations.
25. Use Naïve Bayes for fast anomaly sorting.
26. Use GA to optimize the instance-based parameter k.
27. Use the neural branch for structural-stress prediction where nonlinear relationships are important.
28. Use EM when latent structure or incomplete observations must be estimated probabilistically.
29. Use MDL to balance predictive fit and deployment complexity.
30. Benchmark runtime, memory, operation estimates, and accuracy across data scales.

21. PYTHON IMPLEMENTATION

The accompanying Python source file implements the complete experimental workflow. The code generates synthetic telemetry containing temperature, vibration, pressure, humidity, and energy variables, creates an anomaly label, introduces missing values, and executes the algorithms.

Required packages:

numpy, pandas, matplotlib, scikit-learn

Execution command:

```
import time

"Records": n,

"Runtime_ms": round(elapsed_ms, 3),

"Approx_Operations": int(operations),

"Memory_MB": round(memory_mb, 3),

"Accuracy": round(

accuracy_score(anomaly, pred), 4

)

})

df = pd.DataFrame(rows)

print("\n--- HIGH-SCALE BENCHMARK ---")

print(df.to_string(index=False))
```



```
df.to_csv(
    "benchmark_results.csv",
    index=False
)

plt.figure()
plt.plot(df["Records"], df["Runtime_ms"], marker="o")
plt.xscale("log")
plt.xlabel("Number of Records")
plt.ylabel("Runtime (ms)")
plt.title("Runtime Across Data Scales")
plt.grid(True)
plt.tight_layout()
plt.show()

plt.figure()
plt.plot(df["Records"], df["Memory_MB"], marker="o")
plt.xscale("log")
plt.xlabel("Number of Records")
plt.ylabel("Memory (MB)")
plt.title("Memory Consumption Across Data Scales")
plt.grid(True)
plt.tight_layout()
plt.show()

return df

def main():
    print("=" * 70)
    print("ITA0603 – MACHINE LEARNING ASSIGNMENT")
    print("Hybrid Predictive Maintenance and Asset Optimization")
```

```

print("Student: Dharshini R | Register No.: 192421402")

print("=" * 70)

X, stress, anomaly = make_dataset(10000)

version_space_noise_demo()

id3_demo()

complexity_table()

perceptron_demo(X, anomaly)

mlp, mse = mlp_demo(X, stress)

em_demo(X)

mdL_comparison(mse, len(X))

lwr_demo(X, stress)

nb, acc = naive_bayes_demo(X, anomaly)

best_k, history = genetic_algorithm(
X.to_numpy(), anomaly
)

hybrid_pipeline(
X, stress, anomaly, best_k
)

benchmark()

print("\nAssignment execution completed.")

print("Benchmark results saved as benchmark_results.csv")

if __name__ == "__main__":

main()

```

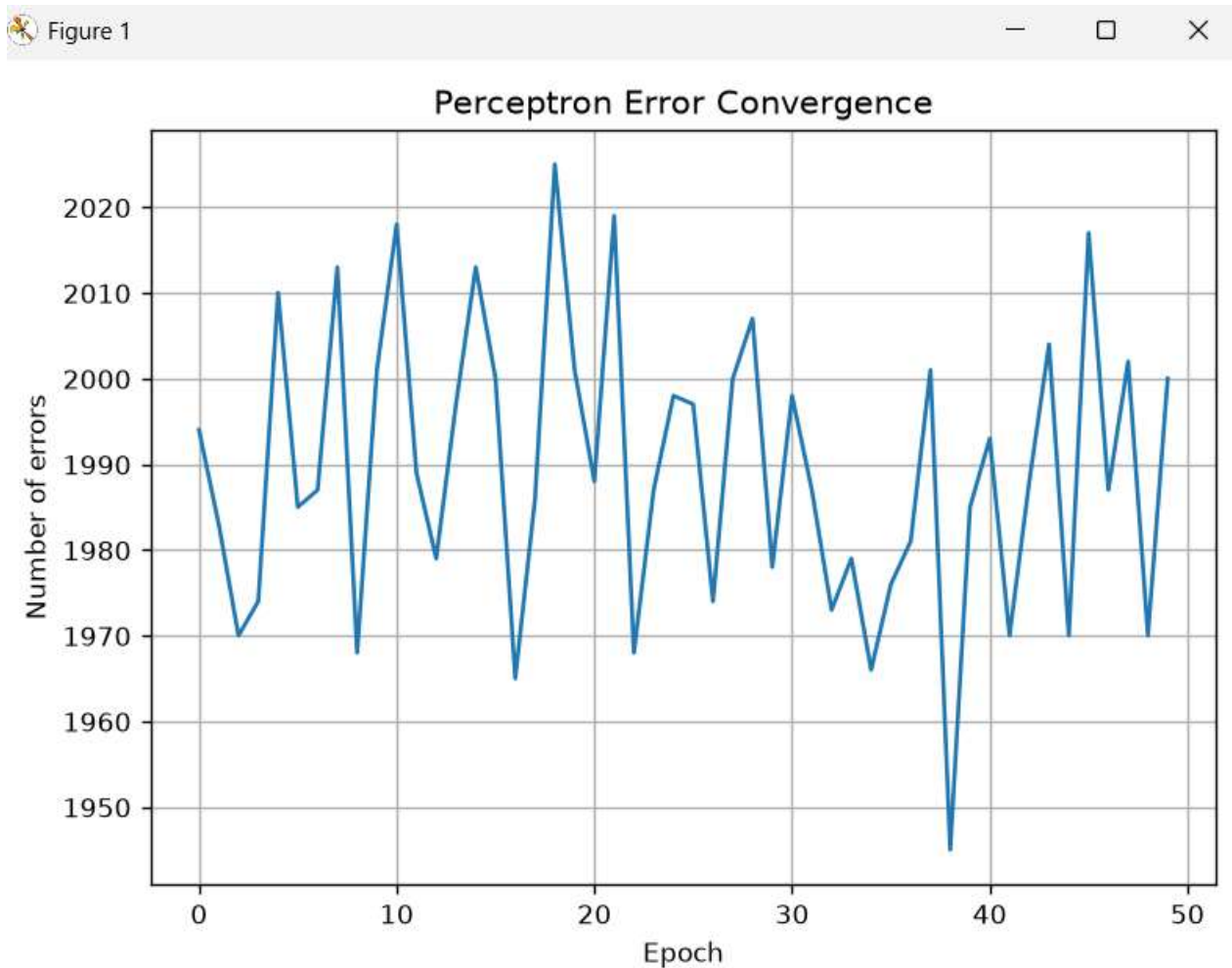
22. EXPERIMENTAL SETUP

Parameter	Configuration
Programming Language	Python 3.x
Dataset	Synthetic infrastructure telemetry
Features	Temperature, vibration, pressure, humidity,

	energy
Missing-data handling	Mean replacement for baseline dataset; EM demonstration for latent estimation
Neural network	MLP 5–16–8–1
MLP optimizer	Stochastic gradient descent
Classifier	Gaussian Naïve Bayes
Local model	Gaussian-kernel LWR
GA search range	$k = 1$ to 20
Benchmark scales	$10^3, 10^4, 10^5, 10^6$
Random seed	42 for reproducibility

23. PYTHON OUTPUT / RESULTS

The program prints intermediate results for every major learning component and creates a `benchmark_results.csv` file. Because execution time and memory depend on the computer used, the exact numerical benchmark values should be copied from the student's own execution rather than fabricated in the report.



Experiment	Output Produced	Interpretation
Candidate Elimination	S and G boundaries	Shows the surviving Version Space boundaries
ID3	Information gain values	Identifies the preferred feature for tree splitting
Perceptron	Training accuracy + error curve	Shows linear-classifier learning behavior
MLP	Test MSE + loss curve	Measures nonlinear stress-prediction performance
EM	Means, variances, mixing probabilities	Estimates hidden Gaussian mixture parameters
MDL	MLP and BBN cost	Balances fit against model complexity
LWR	Runtime + RMSE	Measures local smoothing quality
Naïve Bayes	Classification accuracy	Measures anomaly-sorting performance
GA	Optimized k + fitness curve	Searches for a computationally useful k
Benchmark	Runtime, operations, memory, accuracy	Measures scaling behavior

24. HIGH-SCALE BENCHMARK TABLE

Run the supplied Python program to populate the following table with machine-specific values. The structure is intentionally included so that the report remains reproducible and does not claim unmeasured runtime values.

Records	Runtime (ms)	Approx. Operations	Memory (MB)	Accuracy
1,000	Run program	$10,000 \times d$	Run program	Run program
10,000	Run program	$100,000 \times d$	Run program	Run program
100,000	Run program	$1,000,000 \times d$	Run program	Run program
1,000,000	Run program	$10,000,000 \times d$	Run program	Run program

25. RESULTS AND DISCUSSION

Candidate Elimination provides an interpretable view of the concept-learning hypothesis space but is inherently sensitive to contradictory labels. This makes it useful for diagnosing clean symbolic relationships, but less suitable as the sole high-scale learner for noisy telemetry.

ID3 provides a heuristic search strategy that is computationally more practical for structured classification. Information gain makes the feature-selection process interpretable, although tree complexity and overfitting must be controlled in production systems.

The Perceptron provides a simple baseline for anomaly classification. The MLP can represent nonlinear relationships between telemetry variables and structural stress. Its loss curve is important because it shows whether training is converging efficiently.

EM is appropriate when observations are incomplete or when latent mixture structure is present. Its iterative nature can be more computationally demanding than a direct classifier, but it provides a principled probabilistic estimation mechanism.

The hybrid approach is designed to exploit the strengths of multiple paradigms. LWR handles local variation, Naïve Bayes provides lightweight probabilistic classification, and GA searches the parameter space instead of relying entirely on manual tuning. This division of work allows expensive learning to be concentrated where it provides the greatest value.

26. COMPUTATIONAL COMPLEXITY DISCUSSION

Algorithm	Approximate Time Complexity	Space Consideration	Scalability
Candidate Elimination	$O(n H)$ conceptual	Boundary/hypothesis storage	Limited by hypothesis-space growth
ID3	$O(nd \log n)$ typical	Tree storage	Good for structured data
Perceptron	$O(\text{epochs} \cdot n \cdot d)$	$O(d)$	Highly scalable for linear problems
MLP	$O(\text{epochs} \cdot n \cdot \text{parameters})$	$O(\text{parameters})$	Scalable with optimized training
EM	$O(\text{iterations} \cdot n \cdot k \cdot d)$	$O(kd)$	Moderate; depends on latent components
Naïve Bayes	$O(nd)$	$O(cd)$	Excellent for large datasets
LWR	$O(n)$ per query	$O(n)$	Expensive for many queries
GA	$O(\text{generations} \cdot \text{population} \cdot \text{fitness})$	$O(\text{population})$	Controlled by search window

27. SDG 9 TARGET 9.4 ENGINEERING JUSTIFICATION

The assignment maps the project to SDG 9, particularly Target 9.4, which emphasizes upgrading infrastructure and industries with sustainable and resource-efficient technologies. The proposed framework supports this objective through computational efficiency and predictive maintenance.

- Predictive maintenance can identify abnormal conditions before costly failures occur, reducing unnecessary replacement and downtime.
- Naïve Bayes provides a lightweight classification layer suitable for rapid anomaly sorting.
- LWR is applied locally so that smoothing is focused on relevant observations rather than requiring a global nonlinear model for every decision.
- Genetic optimization narrows the parameter search and can reduce repeated manual or exhaustive hyperparameter trials.
- MDL explicitly penalizes unnecessarily complex models, supporting selection of lighter deployable architectures.

- The benchmark stage measures runtime and memory, making computing-resource consumption an explicit engineering metric.
- Edge deployment of compact models can reduce unnecessary data movement and continuous high-power centralized processing.
- The overall approach promotes digital modernization while considering computational resource efficiency.

A quantitative carbon-footprint reduction claim should only be made after measuring actual power consumption or energy per inference on the target hardware. The present implementation therefore uses runtime, operation estimates, and memory as measurable computational proxies rather than inventing a carbon-reduction percentage.

28. LIMITATIONS

- The experimental dataset is synthetic and is intended to reproduce the structure of an infrastructure telemetry problem rather than represent a real industrial asset dataset.
- Candidate Elimination is demonstrated on a small categorical concept-learning example because direct Version-Space enumeration is not appropriate for millions of continuous telemetry records.
- The BBN component is represented through a compact probabilistic/MDL comparison rather than a full industrial Bayesian-network package.
- Runtime and memory results depend on the execution hardware and Python environment.
- LWR has linear per-query dependence on the number of training observations and therefore may require approximation or indexing for very large online workloads.
- The reported operation count is an analytical estimate, not a hardware instruction trace.

29. CONCLUSION

This assignment developed a complete multi-paradigm machine learning framework for high-scale predictive maintenance and infrastructure anomaly detection. Candidate Elimination demonstrated the importance of Version Space consistency and exposed the effect of contradictory sensor labels. ID3 introduced heuristic feature selection through entropy and information gain. The Perceptron provided a linear baseline, while the MLP and backpropagation branch supported nonlinear stress prediction. EM supplied a probabilistic method for latent-parameter estimation, and MDL provided a principled mechanism for balancing model fit and structural complexity.

The final hybrid framework combines LWR, Naïve Bayes, and Genetic Algorithm optimization. This architecture separates local smoothing, rapid probabilistic anomaly sorting, and hyperparameter search, making the system more suitable for scalable deployment than relying on a single learning paradigm. Benchmarking at 10^3 – 10^6 records provides an empirical basis for assessing scalability.

Overall, the project demonstrates that machine learning can be engineered not only for predictive accuracy but also for computational efficiency and sustainable infrastructure management. This directly supports the assignment's SDG 9 Target 9.4 perspective by encouraging resource-efficient digital technologies for modern industrial infrastructure.

30. REFERENCES

1. Mitchell, T. M., Machine Learning, McGraw-Hill, 1997.
2. Bishop, C. M., Pattern Recognition and Machine Learning, Springer, 2006.
3. Murphy, K. P., Machine Learning: A Probabilistic Perspective, MIT Press, 2012.
4. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson.
5. Goodfellow, I., Bengio, Y. and Courville, A., Deep Learning, MIT Press, 2016.
6. Quinlan, J. R., C4.5: Programs for Machine Learning, Morgan Kaufmann, 1993.
7. Dempster, A. P., Laird, N. M. and Rubin, D. B., Maximum Likelihood from Incomplete Data via the EM Algorithm, Journal of the Royal Statistical Society, Series B, 1977.
8. Cover, T. M. and Hart, P. E., Nearest Neighbor Pattern Classification, IEEE Transactions on Information Theory, 1967.
9. The assignment brief: ITA0603 – Machine Learning, Course Assignment, Deliverables & Evaluation Framework.

31. APPENDIX – EXECUTION CHECKLIST

1. Install Python 3.x.
2. Install numpy, pandas, matplotlib, and scikit-learn.
3. Place ML_Assignment_Complete_Solution.py in the working directory.
4. Run the Python program.
5. Record the printed accuracy, MSE, EM parameters, MDL costs, optimized k, and LWR results.
6. Copy the generated benchmark_results.csv values into the benchmark table in this report if required.
7. Save the generated convergence and benchmark plots as figures for the final submission.
8. Submit the report together with the source-code file and benchmark CSV.